

Docker Commands — Cheatsheet (Lessons 5 & 6)

Quick reference for everything covered in Docker Intro (Lesson 5) and Docker Advanced: Volumes, Networking & Snapshots (Lesson 6). Grouped by topic, not by when it was taught.

Images

```
docker pull <image>           # download an image
docker images                  # list images on this machine
docker rmi <image>             # remove an image
docker build -t <name>:<tag> .   # build an image from a Dockerfile in the current dir
docker tag <image> <newname>:<tag> # add another tag to an existing image
docker history <image>         # show each layer and the command that created it
```

Containers — lifecycle

```
docker run <image>              # create + start a container, foreground
docker run -d <image>           # ...detached (background)
docker run -it <image> bash     # ...interactive, with a shell attached
docker run --name <name> <image> # give it a memorable name instead of a random one
docker run --rm <image>         # auto-remove the container when it exits

docker ps                       # list RUNNING containers
docker ps -a                    # list ALL containers, including stopped

docker stop <container>        # graceful stop
docker start <container>       # start a stopped container (state/data preserved)
docker restart <container>     # restart a stopped container
docker rm <container>          # remove a stopped container
docker rm -f <container>       # force-remove, even if running
```

Containers — interacting

```
docker logs <container>        # see its stdout/stderr
docker logs -f <container>     # ...and follow live
docker exec -it <container> bash # get an interactive shell INSIDE a running container
docker exec <container> <cmd>   # run one command inside it, no interactive shell
docker inspect <container>      # full JSON metadata – IP, mounts, env, everything
docker inspect <container> --format '{{.NetworkSettings.Networks}}' # just one field
```

Volumes (Lesson 6, Block 2)

```
docker volume create <name>           # create a named volume
docker volume ls                       # list volumes
docker volume inspect <name>          # see its real path on the host (Mountpoint)
docker volume rm <name>               # delete it – only works if unused
docker volume prune                   # delete ALL volumes not attached to any container

docker run -v <volume>:<container-path> <image>      # NAMED VOLUME – Docker manages it
docker run -v <host-path>:<container-path> <image>   # BIND MOUNT – you pick the host path
docker run --tmpfs <container-path> <image>        # RAM-only, never touches disk
```

Snapshots (Lesson 6, Block 3)

```
docker commit <container> <newimage>:<tag>  # freeze a container's CURRENT state into an image
                                              # (debugging tool – NOT a substitute for a Dockerfile)

docker save <image> -o file.tar              # package an IMAGE (all layers + tags) to a tarball
docker load -i file.tar                     # restore that exact image, tag included
```

Which one when:

Need	Command
Quickly freeze a broken/tinkered container to inspect later	<code>commit</code>
Move an image to a machine with no registry access	<code>save</code> / <code>load</code>
Flatten many layers into one, don't care about history	<code>export</code> / <code>import</code>

Networking (Lesson 6, Block 4)

```
docker network create <name>           # create a user-defined bridge network (has embedded DNS)
docker network ls                       # list networks
docker network inspect <name>          # subnet, connected containers
docker network rm <name>               # delete a network – must be unused first
docker network connect <network> <container> # attach a running container to another network
docker network disconnect <network> <container>

docker run --network <name> <image>    # attach a container to a network at creation time
docker run --network host <image>      # share the host's network stack directly, no isolation
docker run --network none <image>      # no networking at all
docker run -p <host-port>:<container-port> <image> # publish a port to the host/outside world
```

The six drivers, one line each:

Driver	One-liner
<code>bridge</code> (default)	Isolated per-host network, no name-based DNS
<code>bridge</code> (user-defined)	Same, but with embedded DNS — containers resolve each other by name
<code>host</code>	No isolation — container uses the host's network directly
<code>overlay</code>	Spans multiple Docker hosts — Swarm/multi-host only
<code>macvlan</code>	Container gets its own MAC, looks like a real LAN device — often blocked by cloud providers
<code>ipvlan</code>	Like macvlan, lighter — shares the host's MAC
<code>none</code>	No network access at all

Cleanup / disk usage

```

docker system df                # how much disk Docker is using right now
docker container prune          # remove all stopped containers
docker image prune              # remove dangling (untagged) images
docker image prune -a          # remove ALL images not used by a running container
docker volume prune            # remove unused volumes
docker network prune           # remove unused networks
docker system prune            # all of the above except volumes
docker system prune -a --volumes # ⚠ everything, including volumes – the nuclear option

```